Laboratorio 3

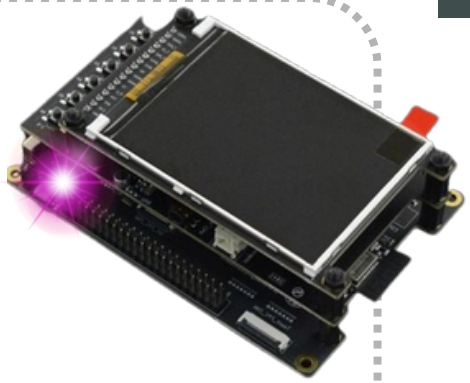
MAIN_APP

TASK_FLOW

TASK_A

```
#include "shared_types.h"
#include "rgb_led.h"
....
```

void task_a(void *pvParameters)



shared_types.c

SemaphoreHandle_t g_color_mutex

shared_types.c

rgb_color_t **g_current_color**

send_status(void)

TASK_C

- same_color(rgb_color_t a, rgb_color_t b)
- remove_pending_timer(TimerHandle_t timer)
- cancel_previous_timer_same_color(...)
- timer_callback(TimerHandle_t xTimer)
- register_pending_timer(...)

void task_c(void *pvParameters)

xQueueReceive

shared_types.c

led_queue

1.led_queue 1
2.led_queue 2
3.led_queue 3
4.led_queue 4
5.led_queue 5
6....

xQueueSend

TASK_B

```
#include "driver/uart.h"
#include "shared_types.h"
...
```

void task_b(void *arg)

send_status(void)

UART_echo

UART

TERMINAL
VIOLETA 10
AMARILLO 10
.....

TASK_A

- Lee el color actual

```
xSemaphoreTake(g_color_mutex, portMAX_DELAY)
local_color = g_current_color;
xSemaphoreGive(g_color_mutex);
```

- Prende el LED

```
rgb_led_set_color(local_color.r, local_color.g, local_color.b);
```

- Espera 500mS

```
vTaskDelay(pdMS_TO_TICKS(BLINK_ON_TIME_MS));
```

- Lo apaga

```
rgb_led_off();
```

- Espera otros 500mS

```
vTaskDelay(pdMS_TO_TICKS(BLINK_OFF_TIME_MS));
```

TASK_B

- **Configura y Inicializa UART**

```
#include "driver/uart.h"

ESP_ERROR_CHECK
uart_driver_install()
uart_param_config()
uart_set_pin()

uart_read_bytes()
uart_write_bytes()
```

- **Procesa la entrada**

```
if (len > 0)

if (byte_recibido == '\n' || byte_recibido == '\r')

for (int i = 0; i < line_index; i++) {
    if (line_buffer[i] == ' ') {

led_command_t led_cmd;

if (strcmp(color_str, "ROJO") == 0) {
 led_cmd.color.r = 255;
   ...
   ...
} else {
        ESP_LOGW(TAG, "Colorb desconocido: %s", color_str);
        color_valido = false;
```

- **Envia los datos**

```
if (color_valido && espacio_idx != -1 && atoi(nptr: (char *)(line_buffer + espacio_idx + 1)) >= 1)

led_cmd.delay_s = delay_s;

xQueueSend(led_queue, &led_cmd, portMAX_DELAY);

else if (line_index < (BUF_SIZE - 1)) {
    line_buffer[line_index++] = byte_recibido;
```

TASK_C

- **Callback del timer “One-shot”**

```
static void timer_callback(TimerHandle_t xTimer)

rgb_color_t *color = (rgb_color_t *)pvTimerGetTimerID(xTimer);

if (xSemaphoreTake(g_color_mutex, portMAX_DELAY) == pdTRUE) {
    g_current_color = *color;
    xSemaphoreGive(g_color_mutex);

   vPortFree(color);
   xTimerDelete(xTimer, 0);
```

- **Espera de comandos en la cola**

```
QueueHandle_t queue = (QueueHandle_t)pvParameters;
led_command_t command;

while (1) {
    if (xQueueReceive(queue, &command, portMAX_DELAY) == pdTRUE)
```

- **Manejo de los comandos recibidos**

```
rgb_color_t *color = (rgb_color_t *)pvPortMalloc(sizeof(rgb_color_t));
 *color = command.color;

TickType_t period = pdMS_TO_TICKS(command.delay_s * 1000);

TimerHandle_t timer = xTimerCreate(
    "led_timer",
    period,
    pdFALSE,
    (void *)color,
    timer_callback
);
```

OPCIONALES

- Status

```
snprintf(
    status_msg,sizeof(status_msg),
    "STATUS: R=%u G=%u B=%u | timers pendientes=%lu\r\n",
    local_color.r,local_color.g,local_color.b,
    (unsigned long)g_pending_timers
uart_write_bytes(UART_NUM_0, status_msg, strlen(status_msg));
```

- Cola con timeout

```
if (xQueueSend(led_queue, &led_cmd, pdMS_TO_TICKS(100)) == pdTRUE) {
    ESP_LOGI(TAG, "Comando enviado a la cola correctamente");
} else {
    ESP_LOGW(TAG, "No se pudo enviar el comando: cola llena");
}
```

- Monitor de stack

```
ESP_LOGI(
        TAG,
        "Stack minimo libre TASK B: %u words",
        (unsigned int)uxTaskGetStackHighWaterMark(NULL)
    );
```

- Cancelación de timers pendientes

```
typedef struct {
    bool active;
    rgb_color_t color;
    TimerHandle_t timer;
    rgb_color_t *color_ptr;
} pending_timer_t;

static pending_timer_t pending_timers[QUEUE_LENGTH];
```

```
for (int i = 0; i < QUEUE_LENGTH; i++) {
    if (pending_timers[i].active &&
        same_color(pending_timers[i].color, cmd.color)) {

        xTimerStop(pending_timers[i].timer, 0);
        xTimerDelete(pending_timers[i].timer, 0);

        vPortFree(pending_timers[i].color_ptr);

        pending_timers[i].active = false;
        g_pending_timers--;

        ESP_LOGI(TAG, "Timer anterior del mismo color cancelado");
    }
}
```

APP_MAIN

```
void app_main(void)
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "shared_types.h"
#include "rgb_led.h"
#include "task_b.h"
#include "task_c.h"
#include "task_a.h"
```

Setup

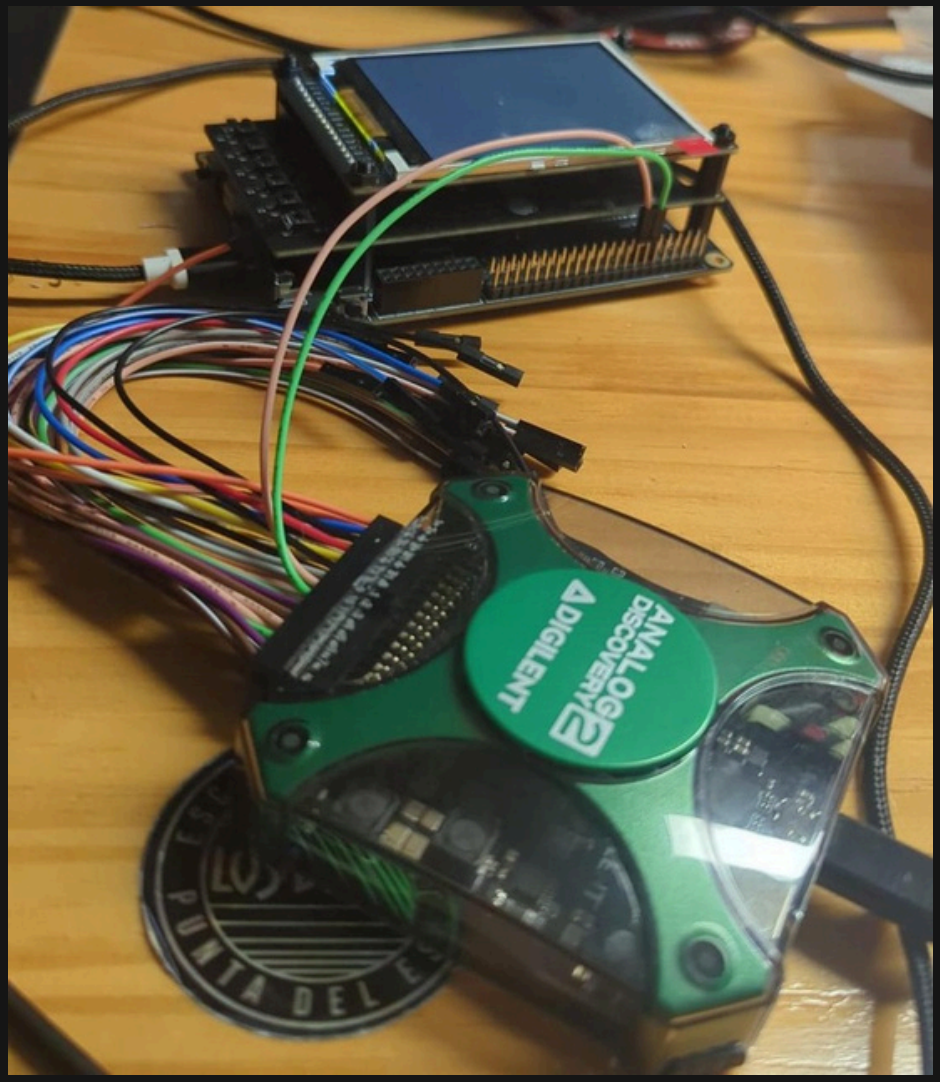
- xQueueCreate()
- g_color.r = 255;
  g_current_color.g = 0;
  g_current_color.b = 0;
- g_pending_timers = 0
- xSemaphoreCreateMutex();
- rgb_led_init();

Task Create

```
task_created = xTaskCreate(
    pxTaskCode: task_a,
    pcName: "task_a",
    usStackDepth: 800,//usa 628 segun sale
    pvParameters: NULL,
    uxPriority: tskIDLE_PRIORITY + 1,
    pxCreatedTask: NULL
);
```

```
task_created = xTaskCreate(
    pxTaskCode: task_b  ,
    pcName: "task_b",
    usStackDepth: 2100,
    pvParameters: (void *)led_queue,
    uxPriority: tskIDLE_PRIORITY + 3,
    pxCreatedTask: NULL
);
```

```
task_created = xTaskCreate(
    pxTaskCode: task_c,
    pcName: "led_task",
    usStackDepth: 1050,
    pvParameters: (void *)led_queue,
    uxPriority: tskIDLE_PRIORITY + 2,
    pxCreatedTask: NULL
);
```


ANALOG DISCOVERY 2
DIGILENT